

Project Blueprint: Kinodynamic Target Interception (The "Robot Soccer Striker")

1. The Mathematical Formulation (For your Report)

To get full marks for "Technical Depth", your report must clearly define the physics and the optimization constraints. Here is the math you will use:

A. Nonholonomic Car Dynamics (Kinematic Bicycle Model)

The state of the car is $q = [x, y, \theta, v]^T$, and the control inputs are $u = [a, \delta]^T$ (acceleration and steering angle).

- $\dot{x} = v \cos(\theta)$
- $\dot{y} = v \sin(\theta)$
- $\dot{\theta} = \frac{v}{L} \tan(\delta)$ (where L is the wheelbase)
- $\dot{v} = a$

B. The Geometric "Strike" Constraint (The Billiards Math)

To shoot the ball into the goal coordinate $G = (x_g, y_g)$, the car must hit the ball at a specific impact point $P_{impact} = (x_b(T), y_b(T))$ with a specific heading.

- Target Heading (θ_{strike}):** The car must be facing the goal at the exact moment of impact.

$$\theta_{strike} = \arctan 2(y_g - y_b(T), x_g - x_b(T))$$

- Optimal Strike State:** $q_{strike} = [x_b(T), y_b(T), \theta_{strike}, v_{impact}]^T$

C. The Shrinking-Horizon NMPC Formulation

Unlike standard MPC which has a fixed prediction horizon (e.g., always looking 2 seconds ahead), interception requires a **Shrinking Horizon**. If the Neural Network predicts the strike happens at $T = 3.0$ seconds, and your time step $dt = 0.1$, your initial horizon is $N = 30$. At the next step, $N = 29$, counting down to impact. The Cost Function to minimize at any current time step t :

$$J = \underbrace{\|q_N - q_{strike}\|_{Q_N}^2}_{\text{Terminal Cost}} + \sum_{k=t}^{N-1} \underbrace{u_k^T R u_k}_{\text{Control Effort}}$$

Subject to:

- $q_{k+1} = f(q_k, u_k)$ (Dynamics)
- $v_{min} \leq v_k \leq v_{max}$ (Speed limits)
- $-\delta_{max} \leq \delta_k \leq \delta_{max}$ (Steering limits)

2. Project Directory Structure

Set up your Python project exactly like this to keep it organized and professional (it also looks great on GitHub):

```
robot_striker_project/
├── README.md           # Project description and run instructions
├── requirements.txt    # CasADi, PyTorch, NumPy, Matplotlib
├── data/
│   └── strike_dataset.npy # Ignored by git
│                           # Pre-calculated data for NN training
├── models/
│   └── strategy_net.pth  # Saved PyTorch weights
├── src/                # All source code lives here
│   ├── __init__.py
│   ├── simulator.py     # 2D physics and matplotlib rendering
│   ├── data_generator.py # Script to calculate geometric ground-truth
│   ├── network.py       # PyTorch model definition and training loop
│   ├── nmpc_solver.py   # CasADi optimization formulation
│   └── main.py          # The integration loop (runs the show)
└── scripts/
    └── generate_plots.py # Generates the graphs for your final report
```

3. File-by-File Implementation Plan

src/simulator.py

Purpose: Handles the "world". It doesn't do any planning; it just updates the math and draws the pictures.

- `class World` : Holds the car state, ball state, and goal coordinates.
- `def step(car_u)` : Updates the car using Runge-Kutta 4 (RK4) integration on the bicycle model. Updates the ball using $p_{t+1} = p_t + v_t \Delta t$.
- `def render()` : Uses `matplotlib.pyplot` to draw a green rectangle (field), a red dot (ball), a blue rectangle with a line indicating heading (car), and a target area (goal).

src/data_generator.py

Purpose: You need thousands of examples to train your Neural Network. This script generates them using brute-force geometry.

- **Logic:** Spawns the car at $(0, 0)$. Spawns a ball moving in a random direction.
- It iterates through future time T from 0 to 5 seconds. For each T , it checks: "Can the car physically reach the ball's location at time T ?" (e.g., is $\text{distance} / v_{max} < T$).
- It finds the *minimum* T where interception is possible, calculates the required θ_{strike} , and saves the input/output pair to `data/strike_dataset.npy`.
- **Inputs:** `[ball_x, ball_y, ball_vx, ball_vy, car_x, car_y, car_theta]`
- **Outputs:** `[T_strike, x_strike, y_strike, theta_strike]`

src/network.py

Purpose: The Machine Learning "Brain".

- `class StrikeNet(nn.Module) :` A simple 4-layer Multi-Layer Perceptron (MLP). It takes the 7 inputs and outputs the 4 strike parameters.
- `def train() :` Loads `strike_dataset.npy` , splits into train/test, uses MSE Loss and an Adam optimizer, and trains the network, saving the best weights to `models/strategy_net.pth` . (You can run this on your server).

`src/nmpc_solver.py`

Purpose: The mathematical "Spinal Cord" using `CasADi` .

- `class InterceptionMPC :` Initializes the `CasADi Opti()` stack.
- `def solve(current_car_state, target_strike_state, N_steps) :`
 - Creates variables `X` (states) and `U` (controls) of length `N_steps` .
 - Applies the multiple-shooting dynamics constraints $X_{k+1} == f(X_k, U_k)$.
 - Applies the boundary bounds (steering limits).
 - Sets the objective to minimize control effort *and* strictly penalize the distance between X_N and `target_strike_state` .
 - Returns the optimal first control action u_0 .

`src/main.py`

Purpose: The grand finale. This is the file you run to watch the magic happen.

1. Initialize `World` .
2. Load the trained `StrikeNet` `PyTorch` model.
3. Randomly kick the ball.
4. Pass the initial states to `StrikeNet` to get `T_strike` and `q_strike` .
5. **The Loop:** While `time < T_strike`:
 - Calculate remaining steps: `N = (T_strike - current_time) / dt` .
 - Pass `current_car_state` , `q_strike` , and `N` to `InterceptionMPC.solve()` .
 - Apply the returned u_0 (acceleration, steering) to `World.step()` .
 - Call `World.render()` .
6. Print "GOAL!" when the car hits the ball at the exact right angle.

4. Development Timeline (Action Plan)

- **Now to May 24:** Write `simulator.py` and `nmpc_solver.py` . Test them by giving the MPC a *static* target to make sure your `CasADi` math works.
- **May 25 to May 31:** Write `data_generator.py` and let it generate 50,000 examples. Write `network.py` and train it on your server. Verify the network outputs reasonable predictions. (*Submit your project topic*).
- **June 1 to June 7:** Connect everything in `main.py` . This is where you will debug the "Shrinking Horizon" logic. Record a video of a successful run for your Proposal presentation.
- **June 8 to June 15:** Write `generate_plots.py` to extract tracking error metrics and solve times.

.....